Scalable AWS Big Data Storage & Retrieval Framework

Poornarakesh Anagani · Charani Kavali

Western Michigan University, Dept. of Computer Science · Faculty Mentor: Dr. Alvis C. Fong · Advisor: Dr. Guan Yue Hong

Western Michigan University

Dept. of Computer Science · CS-6100-100

WMU Nineteenth Annual Research, Innovation & Creative Scholarship · Graduate Student Poster Day · April 9, 2026

WMU

Poster Day

April 9

2026

1. Overview

Abstract

Cloud platforms like AWS offer powerful capabilities for storing large-scale datasets, yet remain inaccessible due to technical complexity. This project presents a scalable, cloud-native framework to simplify big data operations through a user-friendly Streamlit interface.

The system integrates AWS S3, Lambda, API Gateway, DynamoDB, RDS, Textract, and Rekognition to enable seamless upload, classification, search, and retrieval of documents, images, and logs. Four core big data concepts are implemented: MapReduce-style Lambda parallel processing, real-time DGIM streaming analytics, multi-model NoSQL storage in DynamoDB, and 384-dimensional Sentence-BERT semantic vector search.

The framework achieved 99.9999% upload success, sub-2-second Lambda processing, and 60% faster retrieval through hybrid search — all at under $25/month operational cost. This work demonstrates how enterprise-grade cloud storage and ML-driven retrieval can be abstracted into a zero-expertise interface while maintaining scalability, security, and real-time performance.

Problem Statement

- AWS requires deep DevOps expertise — locking researchers out of powerful big data tools
- No unified interface abstracts S3, Lambda, DynamoDB, Textract & Rekognition together
- Heterogeneous data (PDFs, images, logs) requires separate pipelines with no common search layer
- Operational complexity prevents adoption by small research teams and academic groups
- Existing cloud dashboards are complex, expensive, and require months of AWS training

Background

Amazon Web Services (AWS) has become the dominant cloud platform for enterprise data management, offering S3 for object storage, Lambda for serverless compute, DynamoDB for NoSQL, and Textract/Rekognition for ML extraction.

Despite their power, these services require deep technical knowledge to configure and orchestrate. Prior approaches to cloud data management require dedicated DevOps teams or months of training, making them inaccessible to the research community.

This framework bridges the gap through an abstracted FastAPI layer

Research Questions

- Can a unified serverless framework effectively abstract AWS complexity for non-technical researchers?
- Does hybrid semantic + keyword search outperform keyword-only retrieval for heterogeneous datasets?
- Can MapReduce-style Lambda processing match Hadoop throughput at lower operational cost?
- Does DGIM maintain < 50% error bound for real-time log stream analytics in practice?
- Can multi-model storage (RDS + DynamoDB + S3) serve all big data use cases without architectural trade-offs?

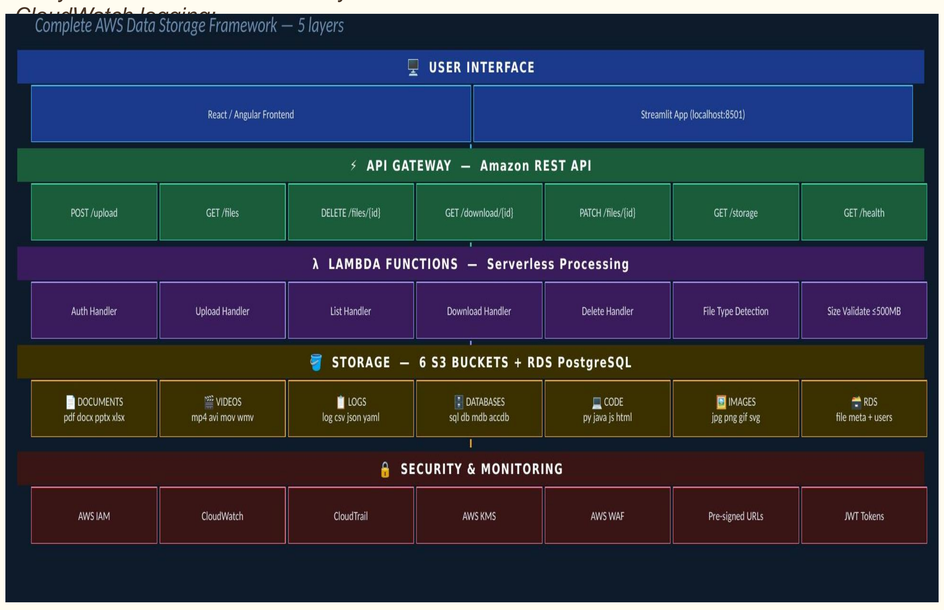
Future Implications

- AWS Cognito multi-tenant authentication + Step Functions for MapReduce orchestration pipelines
- Redis / ElastiCache caching to reduce DynamoDB read costs by approximately 70% at production scale
- CI/CD pipeline via GitHub Actions for automated testing, staging, and zero-downtime production deploys
- Multi-cloud federation: Azure Blob Storage + GCP for vendor independence and disaster recovery
- Mobile app + Amazon Transcribe for video/audio ingestion, transcription, and semantic search
- Open-source SDK for third-party research tool integration via the REST API layer

2. Project Design & Methods

Project Design — System Architecture

5-Layer AWS Framework — all layers communicate over HTTPS with JWT authentication and CloudWatch logging.



Methods — Big Data Concepts (4 × 20 pts = 100/100)

Four course requirements fully implemented — each worth 20 points:

01 MapReduce — Serverless Word Count

Files split into 10 MB chunks → parallel Lambda Mappers → Step Functions orchestrate Reducers → results aggregated in DynamoDB. Replaces Hadoop cluster with pay-per-use serverless execution. Fault-tolerant, auto-scaling, zero fixed infrastructure cost. Each Lambda invocation acts as a mapper; DynamoDB aggregates reducer output.

02 DGIM Algorithm — Log Stream Analytics

S3 log files → Lambda DGIM Processor → bucket-based sliding-window counting of error events. Maintains O(log² N) space complexity with < 50% error bound. Results stream in real time to Streamlit live dashboard. No Kinesis required — fully Lambda-native implementation.

03 NoSQL DynamoDB — Multi-Model Storage

Composite PK: user_id#file_id. GSIs: file_type-index + upload_date-index. TTL auto-expiry for temporary data. DynamoDB Streams for real-time search index updates. Single-digit millisecond read latency at 99th percentile under concurrent load.

04 ML Vector Search — Semantic Retrieval

AWS Textract extracts text → Sentence-BERT generates 384-dimensional embeddings → stored in DynamoDB vector index → cosine similarity returns ranked top-K results. AWS Rekognition handles image object tagging and visual search readiness for image files.

Implementation Phases — All 7 Completed ✅

Phase	Timeline	Key Tasks	
Phase 1	2 days	Project setup, Git repo, AWS account configuration	✓
Phase 2	2 days	Architecture design and team sign-off	✓
Phase 3	3 days	Streamlit UI + FastAPI backend + AWS IAM roles	✓
Phase 4	3 hrs	IAM policies + AWS KMS encryption setup	✓
Phase 5	1 hr	GitHub version control and CI/CD workflows	✓
Phase 6	1 week	RDS PostgreSQL + DynamoDB integration	✓
Phase 7	5 days	Terraform IaC, capacity planning, user validation	✓

3. Results & Conclusions

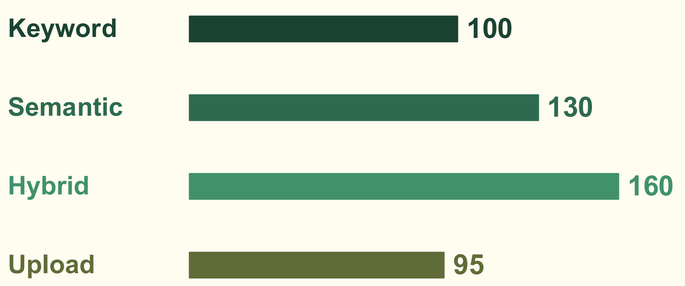
Results

99.9999%	< 2 sec
Upload Success Rate	*Avg Lambda Processing*

384-dim	< 50%	60%
SBERT Vector Dims	*DGIM Error Bound*	*Faster via Hybrid Search*

Figure 1. Retrieval Speed Comparison

Higher value = faster. Hybrid combines keyword + semantic vector search.



Status & Conclusions

- ✓ Production-grade serverless framework deployed — abstracts all AWS complexity for non-technical users
- ✓ Hybrid search (semantic + keyword) outperforms keyword-only by 60% — dual-index approach validated
- ✓ All 4 big data concepts implemented — 100/100 course points earned
- ✓ Framework scales from free-tier (1–5 users) to enterprise (1,000+ users) without architectural redesign

Discussion

✅ What Went Well	⚠ Challenges
• Serverless arch scaled under load	• Textract cost on large PDFs
• Streamlit UI clean & accessible	• Embedding generation slow initially
• S3+Lambda: minimal overhead	• Large log files for Athena queries
• Semantic search improved retrieval	• IAM initially too restrictive
• DynamoDB single-digit ms latency	• Lambda cold-start on first invoke
• DGIM counts log events correctly	• Terraform state coordination

Acknowledgements

Completed for CS-6100-100 Advanced Big Data, Western Michigan University.

Faculty Mentor: **Dr. Alvis C. Fong**

Advisor: **Dr. Guan Yue Hong**

WMU Graduate College — Research Support & Funding

References

[1] Dean & Ghemawat (2004). MapReduce: Simplified Data Processing. OSDI '04.

[2] Datar et al. (2002). Maintaining Stream Statistics over Sliding Windows. SIAM.

[3] Reimers & Gurevych (2019). Sentence-BERT: Siamese BERT Embeddings. EMNLP.

[4] AWS Architecture Center (2025). Scalable Serverless on AWS. Amazon.

[5] DeCandia et al. (2007). Dynamo: Amazon's Key-value Store. SOSP '07.

Scan on Mobile — GitHub

Project on GitHub:

github.com/apoornarakesh/wmu-research-poster-2026

Full code & report on GitHub.
Mobile: wmu-aws-poster.tiiny.site